Comprehensive Internship Reflection Report (Days 1–30)

Candidate: Shri Sanjaykumar V
Academic Institution: Vellore Institute of Technology (VIT), Vellore – M.Tech Integrated in Software Engineering (CGPA: 9.12 / 10.0)
Organization: Linkific – AI / Machine Learning Engineering Internship
Capstone Prototype: FinDoc-AuditEngine v3.0 – Financial Document Reconciliation, Regex PII Masking & Explainable Anomaly Scoring Prototype
Verified Automated Tests: Day 29: 28 passed (python -m pytest tests -q) | Day 30: 33 passed (python -m pytest tests -q) – *all collected tests passed*

1. Executive Summary of the 30-Day Internship Journey

Over the course of this 30-day AI/ML Software Engineering Internship at Linkific, my primary growth was learning how to structure, validate, test, and document an end-to-end applied AI software prototype rather than stopping at isolated notebook experiments.

Drawing motivation from my practical experience managing purchase orders and vendor sponsorships for VIT's Riviera & Gravitas festivals and my prior Generative AI internship at TForce India Private Limited, I focused the final capstone week on building FinDoc-AuditEngine v3.0–a working FastAPI reference prototype for Accounts Payable (AP) auditing. The prototype demonstrates how regex-based pre-inference PII masking, deterministic 3-Way Line-Item Reconciliation (Invoice vs. Purchase Order vs. Goods Receipt Note), a 7-feature StandardScaler + IsolationForest model trained on a reproducible seeded synthetic baseline, and local single-feature perturbation attribution can work together inside a clean, testable web application.

2. 30-Day Technical Progression Summary

Phase	Days	Core Focus Areas	Key Learning & Engineering Deliverables
Phase I: Software & Data Foundations	Days 1–5	Modular Python, Type Annotations, NumPy/Pandas Data Cleaning & Exploratory Analysis	Structured reusable data preprocessing pipelines, statistical outlier handling, and clean CLI entry points.
Phase II: Classical Machine Learning	Days 6–10	Supervised & Unsupervised Learning (scikit-learn), Cross-Validation & Anomaly Detection	Practiced feature scaling, train/validation separation, tree ensembles, and IsolationForest anomaly scoring.
Phase III: NLP & Text Extraction	Days 11–15	Tokenization, Embeddings, Regex Pattern Extraction & Document Classification	Built structured field extractors and text classification pipelines with confidence thresholds.
Phase IV: RAG & Context Grounding	Days 16–20	Vector Indexing, Semantic Retrieval, Context Grounding & Hallucination Guardrails	Explored retrieval-augmented generation workflows and factual-preservation checks.
Phase V: Responsible AI, FastAPI & Testing	Days 21–25	Regex PII Masking, FastAPI Microservices, Pydantic v2 Validation & GitHub Actions CI	Built REST endpoints with strict request schemas, custom error handlers, and automated PyTest suites.
Phase VI: Capstone Prototype & Review	Days 26–30	FinDoc-AuditEngine v3.0 Prototype, Code Review Refactoring, Polish, Slides, Video & Reflection	Integrated the 4-stage AP audit prototype, refined the light-theme UI, recorded the demo walkthrough, and verified all collected PyTests (28 passed in Day 29; 33 passed in Day 30).

3. Technical Skills Acquired

1. Combining Deterministic Domain Rules with Unsupervised ML:
- Learned when to use deterministic logic (ReconciliationMatcher for exact unit-price and dock quantity arithmetic) versus statistical anomaly scoring (AnomalyScorer for surfacing unusual multi-feature combinations such as offshore rush payments).

2. Understanding Synthetic Baselines vs. Production Calibration:
- Designed a reproducible seeded synthetic dataset (N=1,000, random_state=42) to train the prototype's 7-feature IsolationForest and implemented single-feature perturbation attribution ($\Delta R_k = \max(0, R(\mathbf{x}) - R(\mathbf{x}^{(k \leftarrow \mu_k)}))$). Crucially, I learned to distinguish an engineered 0–100 prototype risk indicator from a calibrated production fraud probability.

3. API Design, Input Validation & Automated Testing (FastAPI + Pydantic v2 + PyTest):
- Implemented cross-field validators (@model_validator) to enforce subtotal math within a $0.05 floating-point tolerance, structured domain exceptions (FinDocAuditException), and automated API/unit tests.

4. Pre-Inference Privacy Masking (FinancialPrivacyScrubber):
- Built regex masking for 6 configured patterns (Indian PAN, 12-digit Aadhaar format, bank account numbers, credit cards, phone numbers, and emails) and learned where regex rules succeed and where context-aware NER is required.

4. Major Engineering Challenges Overcome

1. Decoupling Frontend Table Edits from Static Preset Objects:
- *Challenge:* In early iterations of the web dashboard, submitting an audit reused the preset PO's line-item array, which meant manual edits to SKU rates or dock rejected quantities in the browser table did not affect the backend calculation.
- *Solution:* Wrote collectLineItemsFromDOM() in dashboard.js to construct separate InvoicePayload and PurchaseOrderRecord item arrays directly from the live HTML <input> cells in each row, and verified this behavior with a dedicated multi-line regression test.

2. Preventing Dollar Magnitude Dominance in Multivariate Isolation Trees:
- *Challenge:* Raw USD invoice totals ($432 to $49,500) spanned several orders of magnitude compared to bounded variance ratios (0.0 to 1.0).
- *Solution:* Applied logarithmic compression $\ln(1 + \text{Total}_{\text{USD}})$ and standardized all 7 features via StandardScaler prior to fitting IsolationForest.

3. Handling Floating-Point Rounding in Multi-Line Tax Calculations:
- *Challenge:* Standard IEEE 754 binary floating-point products occasionally differed by fractions of a cent, causing false subtotal validation errors.
- *Solution:* Enforced two-decimal rounding at line-item boundaries and configured a $0.05 SUBTOTAL_TOLERANCE across Pydantic validation and 3-way matching.

4. Maintaining Exact Consistency Across Tests, Docs, and Deliverables:
- *Challenge:* Adding new route verification tests increased the test count (28 passed in Day 29 and 33 passed in Day 30), creating a mismatch with earlier documentation and screenshots.
- *Solution:* Audited and synchronized all documentation, PDFs, slides, and terminal screenshots to reflect the exact verified pytest output and honest prototype scope.

5. Key Achievements & Honest Prototype Assessment

- Working End-to-End Prototype (FinDoc-AuditEngine v3.0): Demonstrates 3-way line-item matching, regex PII masking, 7-feature synthetic-baseline anomaly scoring with perturbation explanations, dual-currency (USD/INR) display, and CSV export.
- Verified Automated Test Suite: All 28 collected tests in Day 29 and all 33 collected tests in Day 30 pass via python -m pytest tests -q.
- Clear Documentation of Prototype Limitations: Explicitly documented that AUDIT_LEDGER is an in-memory Python list (non-persistent across restarts), the IsolationForest uses a synthetic baseline rather than real company invoices, and /api/v1/audit/parse-raw performs regex text parsing rather than optical OCR.

6. Core Lessons Learned

1. Precision in Technical Communication Matters as Much as Code: Overstating a prototype as a "deployed production platform" or calling a synthetic baseline "historical company data" undermines credibility during technical review. Clearly stating assumptions, test methods, and limitations builds engineering trust.
2. Automated Tests Must Evolve with the Codebase: Whenever new endpoints or features are added, test counts and documentation should be re-verified immediately before packaging deliverables.
3. UI Clarity Supports Technical Demonstrations: Replacing decorative dark-mode styling with a clean, high-contrast tabular layout made it much easier to demonstrate SKU rate variances and risk attributions during live walkthroughs.

7. Summary of the 90-Day Personal Learning Roadmap

To transition the skills practiced during this internship toward production-grade engineering over the next three months (detailed week-by-week in [PERSONAL_LEARNING_ROADMAP_90_DAYS.md](./PERSONAL_LEARNING_ROADMAP_90_DAYS.md)):
- Month 1 (Days 1–30 – Database Persistence, Auth & Optical Document Parsing): Replace the in-memory AUDIT_LEDGER with PostgreSQL (SQLAlchemy 2.0 + asyncpg) and Redis idempotency keys; add OAuth2/JWT authentication; integrate optical OCR / layout-aware PDF extraction.
- Month 2 (Days 31–60 – Applied ML Evaluation, Calibration & MLOps): Practice probability calibration, precision-recall evaluation on public benchmark datasets, MLflow experiment tracking, EvidentlyAI drift monitoring, and pytest-cov coverage gating in Docker/CI pipelines.
- Month 3 (Days 61–90 – Hybrid RAG, Stateful Agent Workflows & System Design): Build hybrid BM25 + Dense retrieval pipelines with RAGAS evaluation, asynchronous task queues (Celery / Kafka), and practice backend/AI system design for technical interviews.